Microsoft

AI-200T00A

Implement container application hosting on Azure

© Copyright Microsoft Corporation. All rights reserved.

Agenda

- Store and manage containers in Azure Container Registry
- Deploy containers to Azure App Service

© Copyright Microsoft Corporation. All rights reserved.

Module 1: Store and manage containers in Azure Container Registry

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Explain how Azure Container Registry organizes images using registries, repositories, and artifacts
- Build and manage container images in the cloud using ACR Tasks
- Implement tagging and versioning strategies for reliable container deployments
- Use the Azure CLI to manage container images and run ACR quick tasks

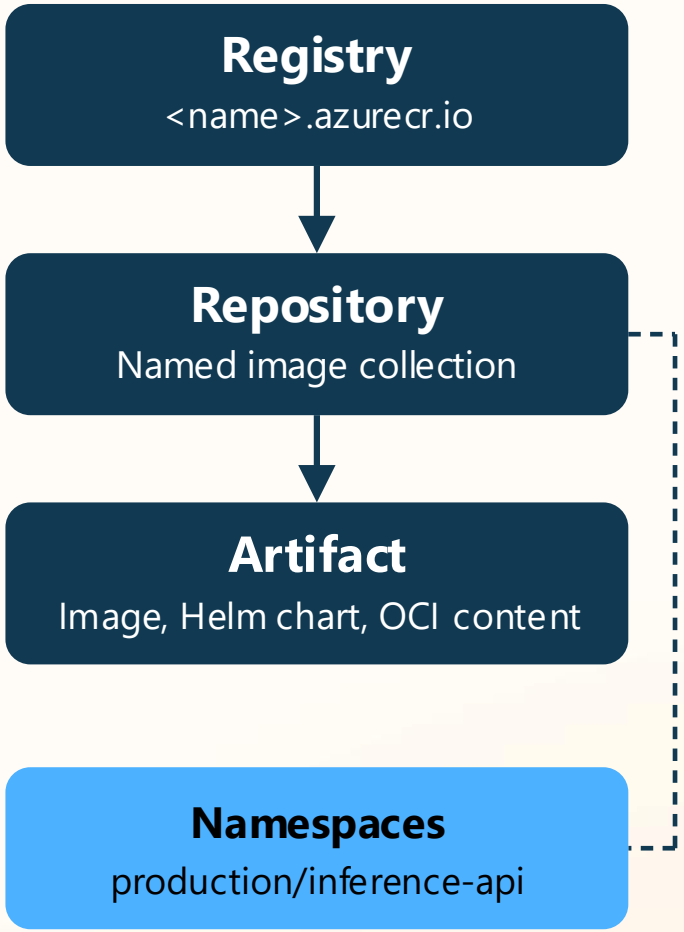
© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Managed, private container registry service on Azure
- Stores and distributes container images and OCI artifacts
- Direct integration with Azure Kubernetes Service, Container Apps, and App Service
- Supports Docker images, Helm charts, and OCI-compliant content
- Basic, Standard, and Premium service tiers
- Geo-replication for global distribution (Premium tier)

© Copyright Microsoft Corporation. All rights reserved.

Registries, repositories, and artifacts



Tags, layers, and manifests

- Tags: Mutable, human-readable references (`repo:tag`). Multiple tags can point to the same artifact.
- Layers: Content-addressable. ACR deduplicates shared layers across images.
- Manifests: Immutable SHA-256 digest (`sha256:abc...`). Pull by digest for guaranteed reproducibility.

Addressing images

- By tag: `registry.azurecr.io/api:v1.2.0`
- By digest: `registry.azurecr.io/api@sha256:0a2e...`

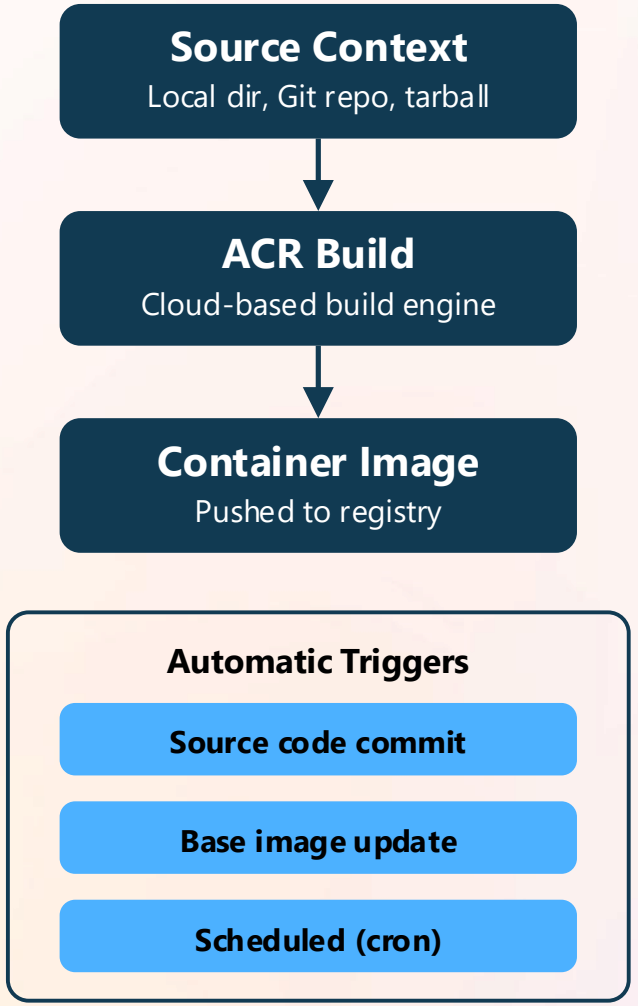
© Copyright Microsoft Corporation. All rights reserved.

Build and run images with ACR Tasks

- Quick tasks build and push in one command
- Source code trigger example

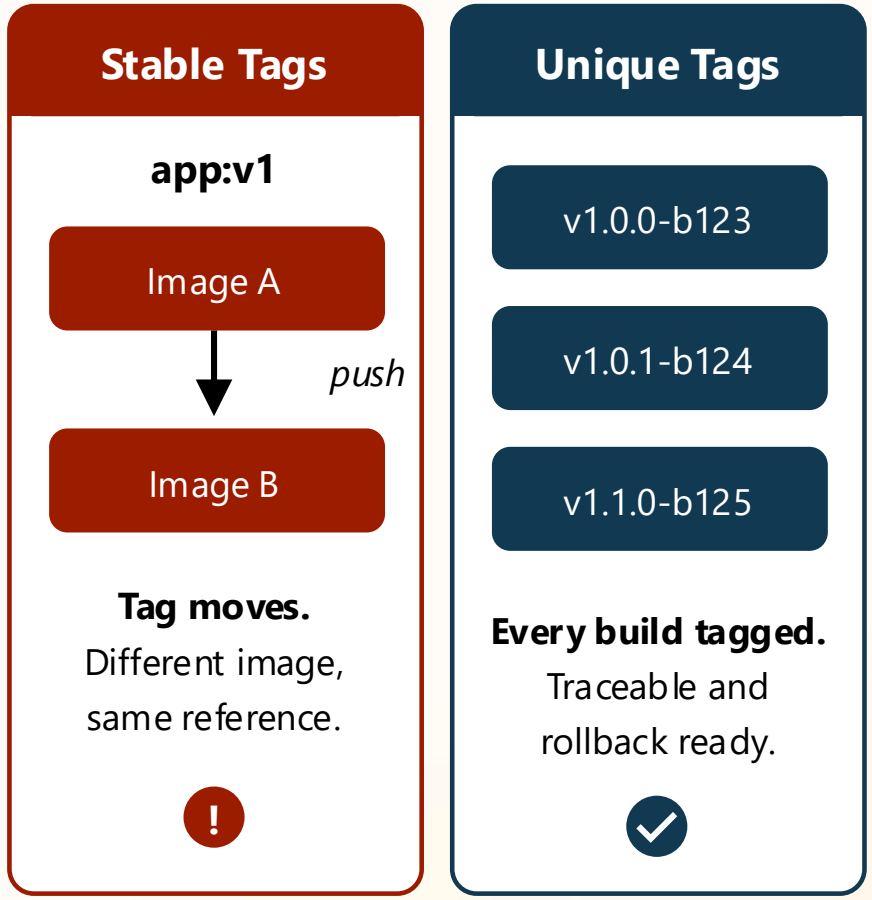
```
az acr task create \
  --registry myregistry \
  --name build-inference-api \
  --image inference-api:{{.Run.ID}} \
  --context https://github.com/myorg/
    inference-api.git#main \
  --file Dockerfile \
  --git-access-token $PAT
```

- Multi-step tasks define build-test-push workflows in YAML



© Copyright Microsoft Corporation. All rights reserved.

Tag and version images



- Lock production images: Prevents deletion, overwrite, and survives retention policies

```
az acr repository update \
  --name myregistry \
  --image inference-api:v1.2.0 \
  --write-enabled false
```

- Clean up untagged images

```
az acr task create \
  --registry myregistry \
  --name cleanup-untagged \
  --cmd "acr purge --filter '.*:.*' \
  --untagged --ago 7d" \
  --schedule "0 0 * * 0" \
  --context /dev/null
```

- Unique tag patterns: build ID, Git SHA, timestamp, or combined

© Copyright Microsoft Corporation. All rights reserved.

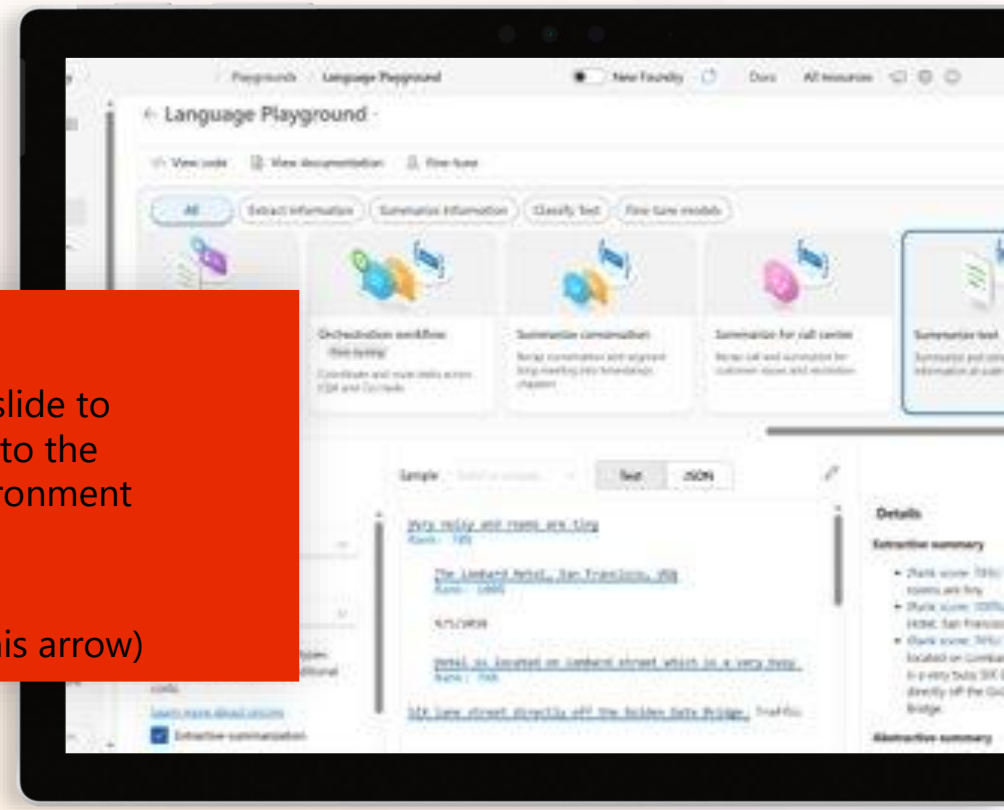
Exercise

Build and run a container image with ACR Tasks

In this exercise, you use Azure Container Registry (ACR) Tasks to build and manage container images entirely in the cloud, without requiring a local Docker installation.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2346552



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 1 knowledge check

1

Your team builds container images on developer workstations, leading to inconsistent results. You need all images built in a controlled environment. Which ACR feature addresses this?

A. ACR Tasks quick build
B. Geo-replication
C. Repository namespaces

2

You need to deploy a container image to production and ensure every node runs the exact same version, even if someone pushes a new image with the same tag. How should you reference the image?

A. By manifest digest
B. By the latest tag
C. By semantic version tag

© Copyright Microsoft Corporation. All rights reserved.

Module 1 summary

- **Registry hierarchy:** Registry, Repository, Artifact with namespace support for organization
- **Tags vs digests:** Tags are mutable references; digests are immutable for guaranteed reproducibility
- **ACR Tasks:** Cloud-based builds via quick tasks, automatic triggers, and multi-step workflows
- **Tagging strategy:** Unique tags for production, stable tags for base images
- **Image locking:** Prevent accidental deletion with `--write-enabled false`
- **Lifecycle management:** Schedule cleanup of untagged images to control storage costs

© Copyright Microsoft Corporation. All rights reserved.

Module 2: Deploy containers to Azure App Service

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Deploy custom containers to Azure App Service from container registries
- Configure container runtime behavior including startup commands, port settings, and persistent storage
- Configure application settings and connection strings in App Service
- Observe and troubleshoot containerized applications using App Service diagnostic tools

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Fully managed platform for custom Linux containers
- Bring your own image from any Docker V2 registry
- Built-in autoscaling, deployment slots, and TLS termination
- Environment configuration via app settings injected as env vars
- Integrated diagnostics: log stream, Kudu console, Azure Monitor
- No infrastructure management. Focus on application code.

© Copyright Microsoft Corporation. All rights reserved.

Deploy containers to Azure App Service

Image sources

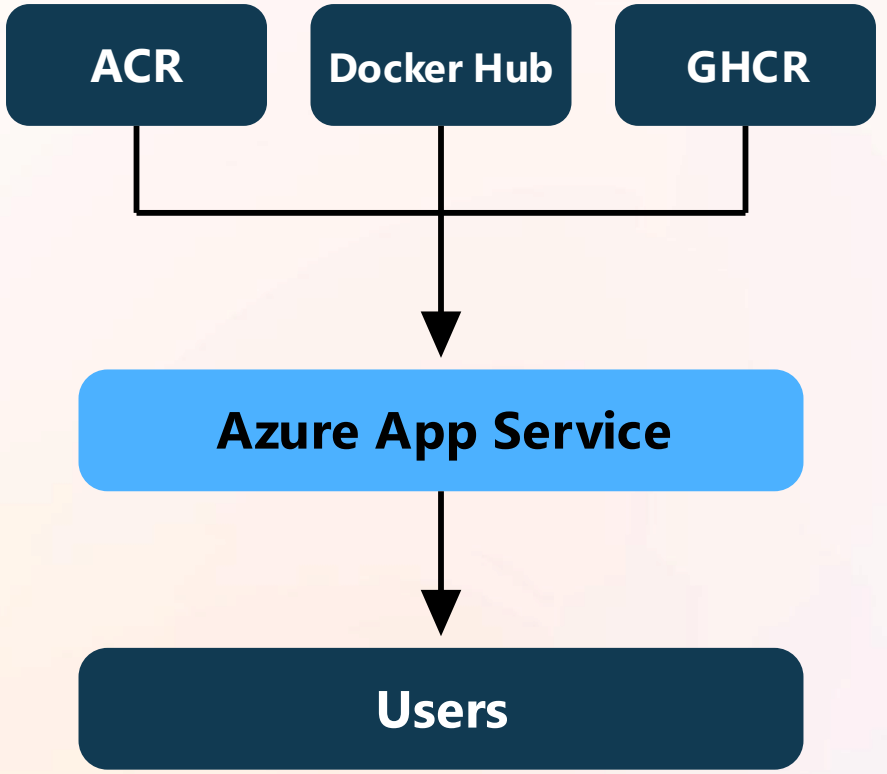
- Azure Container Registry (recommended for production)
- Docker Hub, GitHub Container Registry, self-hosted registries

ACR authentication

- Managed identity (system or user-assigned) with AcrPull role
- Admin credentials for dev/test scenarios

Deployment methods

- Azure portal guided experience
- Azure CLI, VS Code with Docker and App Service extensions
- Continuous deployment via ACR webhook



© Copyright Microsoft Corporation. All rights reserved.

Configure container runtime behavior

Startup command

- Overrides Dockerfile CMD (ENTRYPOINT unchanged)
- Use for runtime args, migrations, custom configs

Port configuration

- Set `WEBSITES_PORT` if container listens on a port other than 80 or 8080
- App Service handles TLS termination. Container receives HTTP.

Persistent storage

- Survives restarts, shared across scaled instances

Always-on and health checks

- Always-on prevents ~20 min idle unloading (Basic tier+)
- Health check path pinged every minute. Failed instances removed from rotation.



© Copyright Microsoft Corporation. All rights reserved.

Configure application settings

App settings

- Injected as environment variables at container start
- All settings encrypted at rest
- Access via standard env var methods (`os.environ` in Python)

Connection strings

- Type-prefixed env vars (e.g., `SQLAZURECONNSTR_`)
- Use app settings instead for non-.NET apps

Slot settings

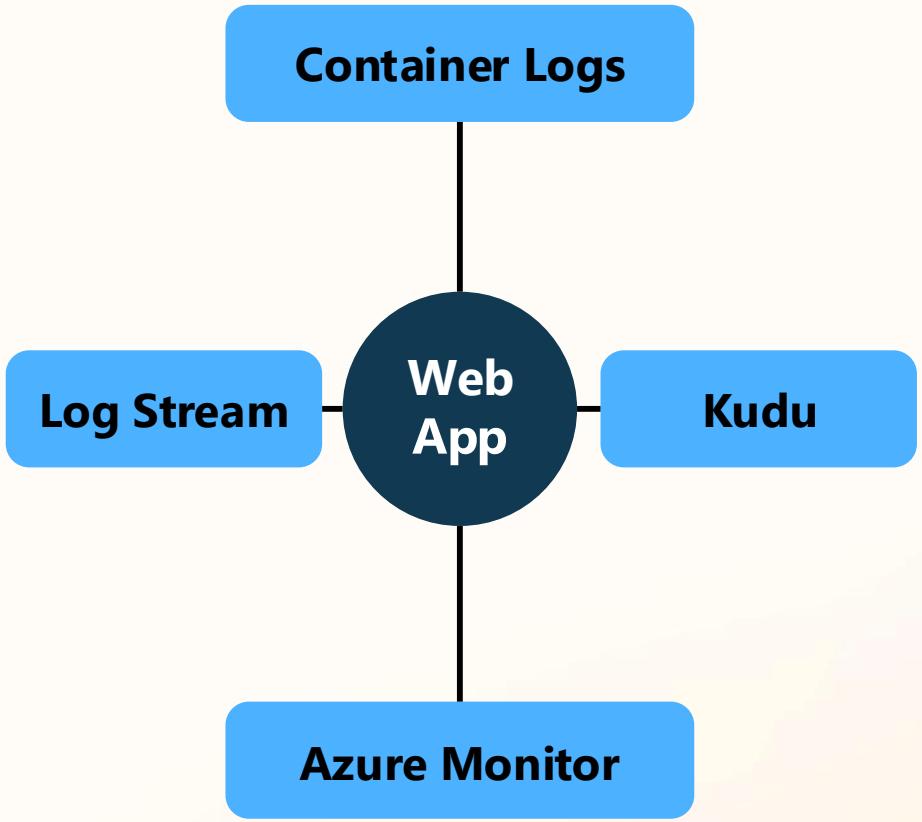
- Stick to the slot during swap operations
- Use for environment identifiers, endpoints, feature flags

Key Vault references

- `@Microsoft.KeyVault(SecretUri=...)` syntax
- Resolved automatically. App reads plain env vars.
- Requires managed identity with Key Vault access

© Copyright Microsoft Corporation. All rights reserved.

Observe and troubleshoot containerized apps



Container logs

- Captures stdout, stderr, platform lifecycle events

Log stream

- Real-time output via CLI (az webapp log tail) or portal
- Shows all instances in scaled-out apps

Kudu (SCM site)

- Environment variable verification
- File system browser for /home/LogFiles/
- Diagnostic dump download

Azure Monitor integration

- Send logs to Log Analytics for KQL queries and alerting
- Categories: ConsoleLogs, HTTPLogs, PlatformLogs

© Copyright Microsoft Corporation. All rights reserved.

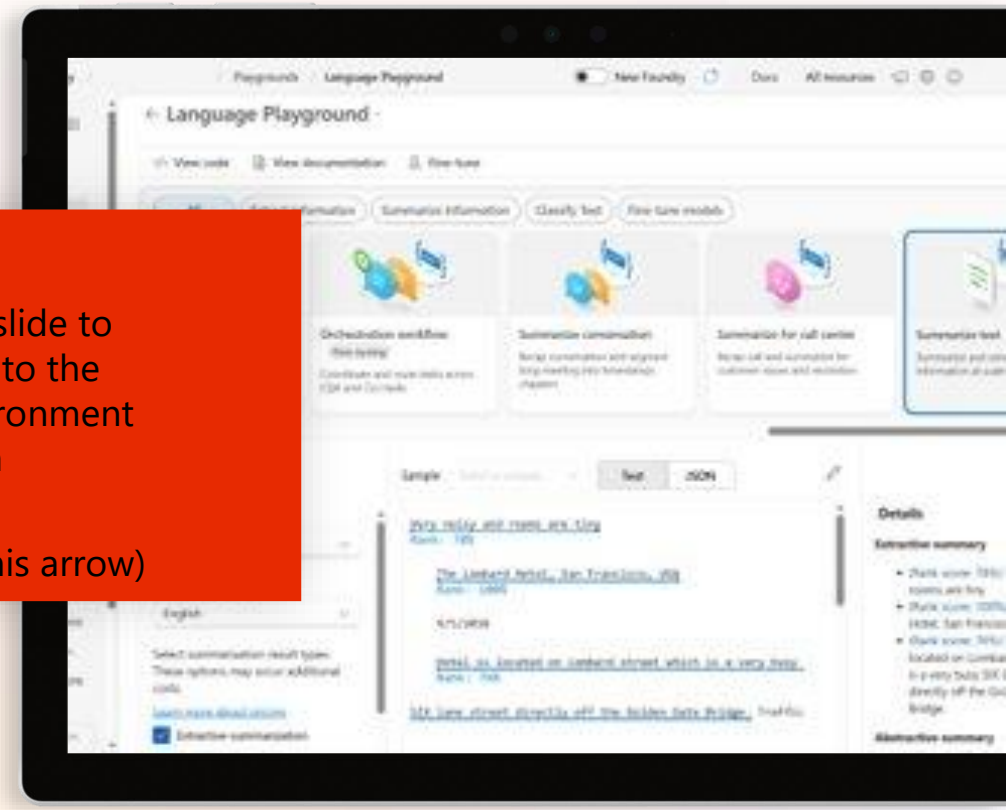
Exercise

Deploy a container to Azure App Service

In this exercise, you deploy a Linux container image from Azure Container Registry (ACR) to Azure App Service.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2347415



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 2 knowledge check

1

A container image listens on port 8000. After deploying to App Service, requests return connection errors. What resolves this?

A. Set the WEBSITES_PORT app setting to 8000
B. Modify the Dockerfile to use EXPOSE 80
C. Enable the HTTP/2 protocol in platform settings

2

A production app needs different API endpoint URLs for staging and production slots. How do you prevent the staging URL from swapping to production?

A. Configure API_ENDPOINT as a slot setting
B. Store the API endpoint in the container image for each environment
C. Use connection strings instead of app settings

© Copyright Microsoft Corporation. All rights reserved.

Module 2 summary

- **Managed identity for ACR:** eliminates credential rotation and is the recommended production auth method
- **WEBSITES_PORT:** must be set when containers listen on ports other than 80 or 8080. Most common deployment issue.
- **Startup command overrides CMD, not ENTRYPOINT:** use it for runtime args, migrations, and custom configs
- **Slot settings:** mark environment-specific values so they stay in the correct slot during swaps
- **Enable container logging explicitly:** stdout and stderr aren't persisted until you turn on diagnostic logging

© Copyright Microsoft Corporation. All rights reserved.

Group discussion

© Copyright Microsoft Corporation. All rights reserved.

Group discussion questions

- Container images go through multiple stages: build, store, deploy, and run. Where in that pipeline have you seen the most issues in practice, and how do ACR features like cloud-based builds, image locking, and digest-based references help reduce risk at each stage?

- App Service injects configuration through environment variables, slot settings, and Key Vault references, while ACR controls what's baked into the image itself. How do you decide what belongs in the container image versus what should be externalized as runtime configuration? Does the answer change for AI workloads that have model paths, API keys, and feature flags?

- Tagging strategy, port configuration, and slot settings are all areas where a wrong default leads to subtle production issues. What processes or guardrails would you put in place to catch these misconfigurations before they reach production, rather than troubleshooting them after deployment?

- As AI applications evolve, teams update base images, swap model versions, and scale across regions. How do you balance the convenience of mutable references (like stable tags and editable app settings) against the reproducibility that immutable references (digests, locked images, versioned configs) provide?

© Copyright Microsoft Corporation. All rights reserved.

End of presentation

© Copyright Microsoft Corporation. All rights reserved.